

Roboty mobilne

Instrukcja do zajęć laboratoryjnych

Ćwiczenie 3. Symulator robota mobilnego

Wstęp

Ćwiczenie rozwija poprzednie zagadnienia związane z programowaniem w Pythonie, wydzielaniem kodu do modułów, testowaniem automatycznym oraz przetwarzaniem danych. Tym razem celem nie jest już tylko odbiór lub filtracja pojedynczego odczytu, ale zbudowanie prostego modelu ruchu robota mobilnego i uruchomienie jego symulacji w przestrzeni 2D.

W ćwiczeniu przyjęto model robota różnicowego opisany przez pozycję x , y oraz orientację θ . Zadanie składa się na implementacji funkcji kinematycznych, symulatorze sekwencji komend ruchu, zapisie trajektorii, utworzeniu wykresu 2D oraz testach jednostkowe i integracyjne.

2. Wymagania wstępne

Do wykonania ćwiczenia potrzebne są biblioteki `matplotlib` oraz `pytest`, które można zainstalować za pomocą polecenia:

```
Pip install matplotlib
```

```
Pip install pytest
```

3. Model robota różnicowego

Robot różnicowy jest robotem mobilnym wyposażonym w dwa niezależnie napędzane koła. Zmieniając prędkość lewego i prawego koła można uzyskać jazdę prostą, skręt po łuku lub obrót w miejscu.

Stan robota w globalnym układzie współrzędnych opisujemy jako:

$$state = (x, y, \theta)$$

gdzie:

- x - położenie robota w osi X [m],
- y - położenie robota w osi Y [m],
- θ - orientacja robota względem osi X [rad].

3.1. Prędkość liniowa i kątowa

Jeżeli znane są prędkości lewego i prawego koła, można wyznaczyć prędkość liniową robota v oraz prędkość kątową ω .

$$v = (v_{right} + v_{left}) / 2$$
$$\omega = (v_{right} - v_{left}) / L$$

gdzie L oznacza rozstaw kół robota.

3.2. Dyskretny model ruchu

Dla małego kroku czasowego dt można zastosować dyskretyzację Eulera. W każdym kroku symulacji wyznaczany jest nowy stan robota.

$$x_{new} = x + v * \cos(\theta) * dt$$
$$y_{new} = y + v * \sin(\theta) * dt$$
$$\theta_{new} = \theta + \omega * dt$$

4. Zadanie 1 - klasa stanu robota

W pliku `robot_state.py` należy przygotować klasę `RobotState` przechowującą pozycję i orientację robota. W tym ćwiczeniu można użyć dekoratora `dataclass`, który upraszcza tworzenie prostych klas danych.

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class RobotState:
    x: float
    y: float
    theta: float
```

```
@dataclass(frozen=True)
class MotionCommand:
    v: float
    omega: float
    duration: float
```

Do wykonania:

- utwórz klasę `RobotState` z polami `x`, `y`, `theta`,
- utwórz klasę `MotionCommand` z polami `v`, `omega`, `duration`,
- zadbaj o adnotacje typów.

5. Zadanie 2 - aktualizacja stanu robota

W pliku kinematics.py należy zaimplementować funkcję `update_robot_state`. Funkcja powinna przyjmować aktualny stan robota, prędkość liniową, prędkość kątową oraz krok czasowy symulacji.

```
def update_robot_state(state: RobotState, v: float, omega: float, dt: float) -> RobotState:
    if dt <= 0:
        raise ValueError("Krok czasowy dt musi być większy od 0.")

    x_new = state.x + v * math.cos(state.theta) * dt
    y_new = state.y + v * math.sin(state.theta) * dt
    theta_new = state.theta + omega * dt

    return RobotState(x_new, y_new, theta_new)
```

Do wykonania:

- sprawdź poprawność parametru `dt`,
- oblicz `x_new`, `y_new` oraz `theta_new` zgodnie z modelem kinematycznym,
- zwróć nowy obiekt `RobotState` zamiast modyfikować istniejący stan.

6. Zadanie 3 - przeliczanie prędkości kół

W robocie różnicowym często steruje się bezpośrednio prędkościami kół. Dlatego należy przygotować funkcję przeliczającą prędkość lewego i prawego koła na prędkość liniową i kątową całego robota.

```
def wheel_speeds_to_velocity(v_left: float, v_right: float, wheel_base: float) -> tuple[float, float]:
    if wheel_base <= 0:
        raise ValueError("Rozstaw kół musi być większy od 0.")

    v = (v_right + v_left) / 2.0
    omega = (v_right - v_left) / wheel_base
    return (v, omega)
```

Do wykonania:

- sprawdź, czy `wheel_base` jest dodatnie,
- policz prędkość liniową `v`,
- policz prędkość kątową `omega`,
- przetestuj przypadek jazdy prosto i skrętu.

7. Zadanie 4 - symulator sekwencji komend

W pliku `simulator.py` należy przygotować funkcję `simulate_commands`, która przyjmuje stan początkowy, listę komend ruchu oraz krok czasowy `dt`. Funkcja powinna zwracać listę kolejnych stanów robota.

```
commands = [  
    MotionCommand(v=0.50, omega=0.00, duration=2.0),  
    MotionCommand(v=0.35, omega=0.60, duration=4.0),  
    MotionCommand(v=0.00, omega=1.00, duration=1.5),  
    MotionCommand(v=0.45, omega=-0.40, duration=3.0),  
]
```

Do wykonania:

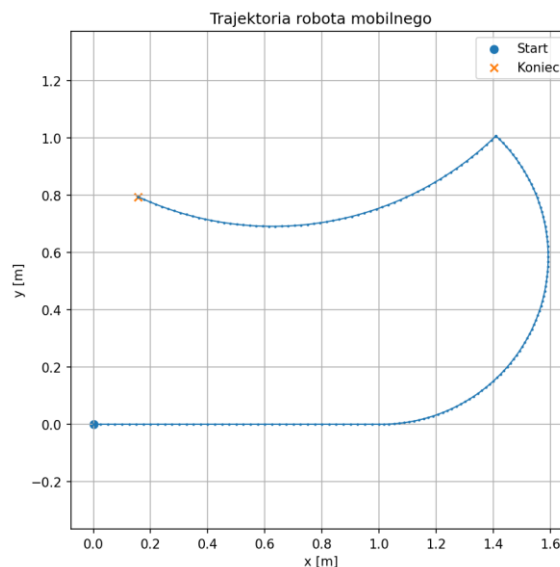
- dla każdej komendy wykonaj odpowiednią liczbę kroków symulacji,
- po każdym kroku zapisz nowy stan robota do listy trajektorii,
- pierwszym elementem trajektorii powinien być stan początkowy,
- obsłuż sytuację, w której czas trwania komendy nie dzieli się dokładnie przez `dt`.

8. Zadanie 5 - zapis i wizualizacja trajektorii

Po wykonaniu symulacji należy zapisać trajektorię do pliku CSV oraz narysować wykres położenia robota w układzie 2D. Wizualizacja pozwala szybko ocenić, czy model ruchu działa zgodnie z oczekiwaniami.

```
trajectory = simulate_commands(initial_state, commands, dt=0.05)  
save_trajectory_csv(trajectory, "results/trajectory.csv")  
plot_trajectory(trajectory, output_path="results/trajectory.png", show=False)
```

Przykładowy wynik działania programu:



Rys. 1. Przykładowa trajektoria robota wygenerowana przez mini-symulator 2D.

9. Zadanie 6 - testy jednostkowe

Testy jednostkowe powinny sprawdzać pojedyncze funkcje w izolacji. W szczególności należy przetestować model ruchu, przeliczanie prędkości kół oraz obsługę błędnych parametrów.

Test	Warunek	Oczekiwany rezultat
Jazda prosto	$v > 0$, $\omega = 0$, $\theta = 0$	rośnie x, y bez zmian
Ruch w osi Y	$\theta = \pi/2$	rośnie y, $x \approx 0$
Obrót w miejscu	$v = 0$, $\omega > 0$	x i y bez zmian, rośnie theta
Błędny dt	$dt \leq 0$	zgłoszony ValueError
Błędny rozstaw kół	$wheel_base \leq 0$	zgłoszony ValueError

```
def test_robot_moves_straight_along_x_axis():
    state = RobotState(x=0.0, y=0.0, theta=0.0)
    new_state = update_robot_state(state, v=1.0, omega=0.0, dt=1.0)

    assert new_state.x == pytest.approx(1.0)
    assert new_state.y == pytest.approx(0.0)
    assert new_state.theta == pytest.approx(0.0)
```

10. Zadanie 7 - test integracyjny

Test integracyjny powinien sprawdzić pełny przepływ obliczeń: od prędkości kół, przez przeliczenie na v i omega, aż do wygenerowania trajektorii robota. Taki test nie sprawdza pojedynczej funkcji, lecz współpracę kilku modułów.

```
def test_full_motion_pipeline_from_wheel_speeds_to_trajectory():
    wheel_base = 0.5
    v_left = 0.3
    v_right = 0.5

    v, omega = wheel_speeds_to_velocity(v_left, v_right, wheel_base)
    commands = [MotionCommand(v=v, omega=omega, duration=1.0)]
    trajectory = simulate_commands(RobotState(0.0, 0.0, 0.0), commands, dt=0.1)

    assert len(trajectory) > 1
    assert trajectory[-1].x > 0.0
    assert trajectory[-1].theta == pytest.approx(omega * 1.0)
```

11. Uruchomienie programu i testów

Symulację można uruchomić poleceniem:

```
python main.py
```

Testy można uruchomić poleceniem:

```
pytest -v
```

Po uruchomieniu programu w folderze results powinny pojawić się pliki:

- trajectory.csv - tabela z kolejnymi stanami robota,
- trajectory.png - wykres trajektorii 2D.

12. Zadanie końcowe dla studenta

W ramach samodzielnego wykonania ćwiczenia student powinien:

1. zmodyfikować sekwencję komend tak, aby robot wykonał co najmniej dwa skręty i jeden obrót w miejscu,
2. wygenerować wykres trajektorii dla własnej sekwencji komend,
3. przygotować minimum trzy testy jednostkowe dla modelu kinematycznego,
4. przygotować minimum dwa testy jednostkowe dla symulatora,
5. przygotować jeden test integracyjny łączący funkcję wheel_speeds_to_velocity z symulatorem,
6. krótko opisać, jak zmiana omega wpływa na kształt trajektorii.

Uwagi końcowe

Ćwiczenie stanowi pomost pomiędzy podstawami programowania i testowania a właściwymi zagadnieniami robotyki mobilnej. Po jego wykonaniu student powinien rozumieć, jak z prostych komend ruchu powstaje trajektoria robota oraz dlaczego wizualizacja i testy automatyczne są ważne przy rozwijaniu algorytmów sterowania.